Contri security audit

Scope: github.com/ohjay-official/contri · commit 17a4a31619c9599a85785995273484a3fed9654b (main)

Contract: circles.contring.near

Date: 6 Nov 2026 (Africa/Lagos)

Methodology: Manual review of contract/src/lib.rs, app/server/*, app/public/js/lib/*, docs/PRIVACY.md, ARCHITECTURE.md, METHODS.md; unit tests as behavior oracles. Attached PDF was SolBlaze content and was ignored.

Counts: Critical 1 · High 2 · Medium 4 · Low 3 · Info 3

> **Safe to remove the owner key now: No.** Critical key-custody risk while any full-access key remains (C1), plus High irrecoverable `paying` / send-callback failure (H1) and a High product gap (fixed one-contribution deposits vs documented invite deposits, H2). Lock only after H1 recovery ships and H2 is implemented or explicitly accepted.

1. Ranked findings

Sev	ID	Title	Location	Impact
Critical	C1	Full-access owner key can drain every pot	`contract account keys`	Whoever holds a full-access key on circles.contring.near can move the contract's entire NEAR Intents USDT balance to themselves, or forge private callbacks to mint claimable credit...
High	H1	paying stuck forever if payout/ send callback fails	`start_payout / finalize_payout / send / transfer_then`	If mt_transfer's callback (on_payout or on_sent) runs out of gas or panics, paying stays true (payout path) or deposits are zeroed with no credit (release/leave/seat path). Keeper ...
High	H2	Early-exit shortfall: deposits fixed at one contribution	`internal_join`	A member who receives an early pot can stop paying. Settle takes at most one contribution from their deposit, then backing, then later recipients take a shortfall. Docs claim invit...
Medium	M1	random_order shuffle is biasable via block random seed	`draw_turns`	Turn order for random_order circles is a Fisher–Yates shuffle over env::random_seed_array(). Chunk producers (and anyone who can choose inclusion timing) can bias who sits first. N...
Medium	M2	Relayer/keeper NEAR drain across serverless instances	`app/server/limit.js; api/relay.js`	allow() is in-memory per isolate. Attackers can fan out IPs/instances to force many simulate+execute or keeper transactions, draining contring.near's NEAR. Signed-intent checks and...

Sev	ID	Title	Location	Impact
Medium	M3	Create-via-Intents griefs contract storage NEAR	`internal_create / ensure_room`	Direct create_circle takes 0.01 NEAR CREATE_FEE. Intents create only locks 1 USDT deposit while the contract pays storage from its own NEAR. Attacker creates many min circles, canc...
Medium	M4	Server learns vault↔circle links on private moves	`confidential.js privateHome / privateMove; PRIVACY.md table row for Contri's server`	privateHome and related handlers receive from (circle account) and vault together. Even without intentional logs, the request body necessarily joins identities the public chain kee...
Low	L1	Privacy unlinkability claims are stronger than the code	`docs/PRIVACY.md`	Each circle account is separate (true for PRF derivation). Linking remains possible via amount/timing correlation (docs partly admit), shared 1Click flows, and server-side private ...
Low	L2	Seat-sale and leave depend on on_sent for seller/leaver funds	`internal_buy_seat / internal_leave / internal_release + on_sent`	State updates (seat swap, deposit cleared) commit before the outgoing mt_transfer is confirmed. Failure + missing callback leaves seller/leaver without tokens and without credit (s...
Low	L3	Device-secret fallback is weaker than passkey PRF	`passkey.js deviceSecret / localStorage`	Where PRF is unavailable, the circle-root secret sits in localStorage (and recovery code). XSS or physical access exports all circle keys. CSP reduces XSS; it is not the same as PR...
Info	I1	Gas floors + GasWeight already used; amounts look adequate	`transfer_then GAS_MT_TRANSFER 15 / GAS_CALLBACK 20 / GAS_FOR_PAYOUT 40; contri.js contribute min_gas 100 Tgas`	No finding that fixed gas alone is unsafe; leftover gas is shared with GasWeight(1). Residual risk is H1 if total attached gas is still too low on worst-case finalize.
Info	I2	owner_id is informational only	`Contri::new / get_config; no assert_owner anywhere`	No in-contract owner backdoor beyond what the account's keys allow. Locking keys removes upgrade/drain; there is no separate owner_withdraw to remove.
Info	I3	Access control of public entrypoints is generally sound	`assert_intents on mt_on_transfer/on_auth; #[private] callbacks; anyone settle/pay_out/release/close`	Non-members cannot join invite circles without a sig; cancel before fill_by is organizer-only; seat list/buy checks are consistent. Keeper privileges are not special on-chain.

2. Finding write-ups

Critical C1 — Full-access owner key can drain every pot

Location: `contract account keys (off-chain) + intents.near mt_transfer as predecessor; #[private] on_sent / on_payout in contract/src/lib.rs`

Impact: Whoever holds a full-access key on circles.contring.near can move the contract's entire NEAR Intents USDT balance to themselves, or forge private callbacks to mint claimable credits.

Attack or failure scenario

1. Sign a transaction as circles.contring.near (full-access key).
2. Call intents.near mt_transfer of nep141:usdt.tether-token.near from the contract's Intents balance to an attacker account (contracts may move their own Intents balance as predecessor).
3. Alternatively: batch a failing promise then callback on_sent with to=attacker and amount=balance; claim_credit drains the same funds via the credit path.
4. owner_id in state is never checked for money movement; the key is the real privilege.

Exact fix: Before lock: rotate to a key you control only for the deploy, verify get_config and live balances, then delete every full-access key on circles.contring.near (keep at most a function-call key if you still need one non-money method). Do not add owner-only withdraw. After lock, #[private] is only reachable via the contract's own promise chain.

High H1 — paying stuck forever if payout/send callback fails

Location: `start_payout / finalize_payout / send / transfer_then (lib.rs ~1331–1436); needs() in app/public/js/lib/contri.js:150; keeper.js`

Impact: If mt_transfer's callback (on_payout or on_sent) runs out of gas or panics, paying stays true (payout path) or deposits are zeroed with no credit (release/leave/seat path). Keeper and needs() skip circles with paying==true, so there is no automatic recovery. Pot can show funds that already left, or money can sit with no claim path.

Attack or failure scenario

1. start_payout sets paying=true, saves, then chains mt_transfer → on_payout.
2. Transfer may succeed or fail on intents.near; the callback receipt then OOGs or panics.
3. State reverts only for the callback receipt: paying remains true; pot is not cleared on the success path if finalize never runs.
4. needs() returns null while paying; settle_round / pay_out require !paying → circle frozen.
5. Same pattern for send(): internal_release / leave / buy_seat / claim_credit zero local balances before on_sent confirms; if on_sent never runs after a failed transfer, no credit is written and amounts are already cleared.

Exact fix: Add an anyone-can-call recover_payout(circle_id) that: require!(paying); require!(block_timestamp > payout_started_at + GRACE); if intents balance still holds the pot amount for this circle's accounting, clear paying and keep pot for pay_out retry; if the transfer already left (track pending_payout_amount + tx intent id, or compare pot vs live Intents balance), run finalize_payout(..., true) idempotently. Persist pending_send receipts for release/leave/seat (do not zero deposit/due until on_sent succeeds, or re-credit on timeout). Change needs() to return recover_payout when paying && grace elapsed. Raise GAS_CALLBACK and keep GasWeight; add a mainnet gas test for size=20 finalize_payout + 20 record writes.

Uncertainty: Storage-reservation tests make a storage panic unlikely; OOG on mainnet is unproven without measuring finalize_payout gas at size=20 under production Intents. Confirm with a sandbox/mainnet profile of on_payout gas used.

High H2 — Early-exit shortfall: deposits fixed at one contribution

Location: `internal_join (lib.rs ~1047–1048); max_deposit / refill; docs PRIVACY.md:64, ARCHITECTURE.md:100, METHODS.md:113`

Impact: A member who receives an early pot can stop paying. Settle takes at most one contribution from their deposit, then backing, then later recipients take a shortfall. Docs claim invite-only circles can require prepaid future rounds; the contract rejects any join amount other than exactly one contribution (test an_invite_cannot_change_the_deposit).

Attack or failure scenario

1. Join a size-10 circle with deposit = contribution (only amount accepted).
2. Hold or buy seat 0; receive ~10× contribution in round 0.
3. Miss later rounds: first miss consumes the single deposit; further misses eat backing then shortfall.
4. Later seats are paid less than a full pot; defaulting only hurts the on-chain Record, not clawback of the early pot.

> **Exact fix:** Either (A) restore per-invite deposits: in internal_join, if invite_key is set, require valid sig over join_message(..., amount) and allow amount in [contribution, max_deposit()], store that deposit; keep open circles at one contribution; update relay paymentProblem; or (B) delete the false docs/tests claims and surface the early-exit risk in-product, and encourage refill + backing. Prefer (A) if the product promise is protection for late seats.

Medium M1 — random_order shuffle is biasable via block random seed

Location: `draw_turns (lib.rs ~1659–1667); internal_join when circle fills`

Impact: Turn order for random_order circles is a Fisher–Yates shuffle over env::random_seed_array(). Chunk producers (and anyone who can choose inclusion timing) can bias who sits first. Not a direct theft of the pot without membership, but breaks the fairness claim.

Attack or failure scenario

1. Organizer creates with order=random.
2. Last join that fills the circle runs draw_turns in that block's seed.
3. A validator or last-joiner strategy that only reveals/includes when the seed maps them to seat 0 improves their odds.

> **Exact fix:** Commit–reveal or use a verifiable random beacon: e.g. store seed_commit = sha256(member_nonces || prev_block_hash) at fill, then draw in a later block with env::random_seed from a subsequent height, or integrate chain signatures / VRF. Document that current random_order is best-effort only until then.

Medium M2 — Relayer/keeper NEAR drain across serverless instances

Location: `app/server/limit.js; api/relay.js (20/min); api/keeper.js GET (2/min); relay.js execute_intents 300 Tgas`

Impact: allow() is in-memory per isolate. Attackers can fan out IPs/instances to force many simulate+execute or keeper transactions, draining contring.near's NEAR. Signed-intent checks and simulate_intents limit what is executed, but failed or refunded paths still cost gas. limit.js itself documents this.

Attack or failure scenario

1. POST /api/relay with many distinct source IPs (or cold isolates) each under per-instance caps.
2. Use cheap auth_call list_seat/unlist_seat as a member, or GET /api/keeper to make the keeper walk circles.
3. Each execute_intents / functionCall burns relayer NEAR until empty → members cannot relay.

> **Exact fix:** Add a shared store (Redis/Upstash) keyed by IP hash and by signer_id with global caps; require Vercel WAF rate limits as claimed; on keeper GET require a cron secret header (CRON_SECRET) and reject public callers; cap keeper work per invocation more tightly when balance is low.

Medium M3 — Create-via-Intents griefs contract storage NEAR

Location: `internal_create / ensure_room (lib.rs); create Transfer path (no CREATE_FEE); cancel + release`

Impact: Direct create_circle takes 0.01 NEAR CREATE_FEE. Intents create only locks 1 USDT deposit while the contract pays storage from its own NEAR. Attacker creates many min circles, cancels (or waits fill_by), recovers USDT via release_deposits; storage NEAR stays consumed until close_circle, and ensure_room eventually DoS's honest creates.

Attack or failure scenario

1. Spam create:<MIN>:2:60:join:-:x with 1 USDT via relayer.
2. cancel as organizer (or wait 14d); release_deposits; close_circle.
3. Repeat until contract NEAR cannot cover reserved+SPARE_BYTES.

> **Exact fix:** Require a NEAR storage deposit on Intents creates (separate ft/NEAR path or raise MIN_CONTRIBUTION and skim a non-refundable storage fee into an owner treasury before lock), or bond N USDT non-refundable on create. Mirror CREATE_FEE economics on the Transfer create path.

Medium M4 — Server learns vault↔circle links on private moves

Location: `confidential.js privateHome / privateMove; PRIVACY.md table row for Contri's server`

Impact: privateHome and related handlers receive from (circle account) and vault together. Even without intentional logs, the request body necessarily joins identities the public chain keeps separate. Docs say the server 'can't' link circles and keeps no IP/account logs — overstated versus this API shape.

Attack or failure scenario

1. Member moves payout home: client calls phome with { from: circleAccount, vault, amount }.
2. Server builds the 1Click quote and ticket; operator or compromised host sees both ids.
3. Repeat across circles → link graph of one passkey's accounts.

> **Exact fix:** Have the client obtain the 1Click quote with a user-held partner key or a blinded relay that never sees both ids in one request; or document clearly that Contri's server is trusted for linkability of private-balance bridges, and encrypt bodies to a seal that only 1Click opens.

Low L1 — Privacy unlinkability claims are stronger than the code

Location: `docs/PRIVACY.md (table + layers 2, 6); keys.js HKDF; oneclick top-ups`

Impact: Each circle account is separate (true for PRF derivation). Linking remains possible via amount/timing correlation (docs partly admit), shared 1Click flows, and server-side private moves (M4). 'Nobody can link your circles' is overstated.

Attack or failure scenario

1. Observe exchange withdrawal $X and circle account credit $X−fee within minutes.
2. Or observe multiple circle accounts funded from the same confidential session patterns.

> **Exact fix:** Rewrite claims to: 'accounts are not key-linked on chain; correlation and the relayer/1Click path can still link activity.' Keep the honest paragraph on amount/timing; demote absolute unlinkability.

Low L2 — Seat-sale and leave depend on on_sent for seller/leaver funds

Location: `internal_buy_seat / internal_leave / internal_release + on_sent`

Impact: State updates (seat swap, deposit cleared) commit before the outgoing mt_transfer is confirmed. Failure + missing callback leaves seller/leaver without tokens and without credit (see H1).

Attack or failure scenario

1. buy_seat swaps seats and clears listing, then send(seller, price).
2. Transfer fails and on_sent never runs → seller unpaid, buyer holds earlier seat, price stuck on contract Intents balance.

> **Exact fix:** Same pending-send journal as H1: only clear listing slot / deposit after on_sent success; on failure write credit (already intended) and ensure callback gas.

Low L3 — Device-secret fallback is weaker than passkey PRF

Location: `passkey.js deviceSecret / localStorage`

Impact: Where PRF is unavailable, the circle-root secret sits in localStorage (and recovery code). XSS or physical access exports all circle keys. CSP reduces XSS; it is not the same as PRF non-extractability.

Attack or failure scenario

1. User falls back to device secret; malware or XSS reads localStorage contri.device-secret.
2. Attacker derives every circleSigner and vaultSigner for that contract.

> **Exact fix:** Prefer WebCrypto non-extractable keys wrapped like session.js; warn in-UI that recovery-code mode is hot-wallet equivalent; refuse device mode for circles above a size/amount threshold.

Info I1 — Gas floors + GasWeight already used; amounts look adequate

Location: `transfer_then GAS_MT_TRANSFER 15 / GAS_CALLBACK 20 / GAS_FOR_PAYOUT 40; contri.js contribute min_gas 100 Tgas`

Impact: No finding that fixed gas alone is unsafe; leftover gas is shared with GasWeight(1). Residual risk is H1 if total attached gas is still too low on worst-case finalize.

Attack or failure scenario

1. Last contribute attaches 100 Tgas; contract requires ≥40 Tgas left to start_payout; transfer_then splits remainder between mt_transfer and on_payout.

> **Exact fix:** Keep weights; add CI assertion on gas_used for size=20 on_payout; optionally bump GAS_CALLBACK to 30 Tgas after measuring.

Info I2 — owner_id is informational only

Location: `Contri::new / get_config; no assert_owner anywhere`

Impact: No in-contract owner backdoor beyond what the account's keys allow. Locking keys removes upgrade/drain; there is no separate owner_withdraw to remove.

Attack or failure scenario

1. Call get_config → owner_id set at init; no payable owner method references it.

> **Exact fix:** Optional: zero or rename owner_id after lock for clarity; not required for safety if keys are gone.

Info I3 — Access control of public entrypoints is generally sound

Location: `assert_intents on mt_on_transfer/on_auth; #[private] callbacks; anyone settle/pay_out/release/close`

Impact: Non-members cannot join invite circles without a sig; cancel before fill_by is organizer-only; seat list/buy checks are consistent. Keeper privileges are not special on-chain.

Attack or failure scenario

1. Predecessor checks and seat ownership checks reviewed against METHODS.md.

> **Exact fix:** No code change required for the happy-path ACL; rely on H1 recovery and C1 key removal.

3. Docs vs code — false or overstated claims

Source	Claim	Correction
PRIVACY.md §3	"The invite also sets the deposit, so an organizer can privately ask less of someone they trust and more of a stranger."	internal_join requires amount == contribution.0 always. Test an_invite_cannot_change_the_deposit. join_message still binds deposit in the sig, but non-contribution amounts are refunded.
PRIVACY.md Status	Lists unit tests invite_can_lower_the_deposit_for_someone_trusted and invite_can_ask_a_stranger_to_cover_their_future_rounds	Those tests are not in contract/src/tests.rs; replaced by an_invite_cannot_change_the_deposit.
ARCHITECTURE.md Missed payments	"In an invite-only circle the organizer sets each person's deposit in their invite..."	False vs code. Same file's message table says join deposit is "at most one contribution" — docs disagree with themselves; code is exactly one contribution.
METHODS.md deposits	"In an invite-only circle the invite sets it, up to every round but the member's own"	False; refill can raise deposit up to max_deposit() after join, but join itself cannot.

Source	Claim	Correction
METHODS.md Seats	"A cancelled circle's seats are burned."	Cancelled circles never mint seats (has_seat_tokens only Active|Finished). Nothing to burn.
ARCHITECTURE.md Actors	"A NEAR smart contract with no keys on its account once locked"	Aspirational. Owner key is still present; treat as unlocked.
PRIVACY.md / table	Stranger cannot connect circles / each circle uses its own account	Accounts are not key-linked on chain (true for HKDF). Absolute unlinkability overstated: amount/timing, 1Click, and server privateHome (M4/L1).
PRIVACY.md Contri's server	"It keeps no logs... and it can't read circle names."	App code avoids logging. Platform/Vercel/CDN logs are outside the repo. Server sees account ids in relay and swap bodies.
PRIVACY.md	"your name never touches the chain"	Contract stores name bytes as given (up to 256). App encrypts in invite flow; a client can still put plaintext.

4. Final answers

(1) Ranked findings — see table and write-ups above (C1, H1, H2, M1–M4, L1–L3, I1–I3).

> **(2) Safe to remove the owner key now: No.** Removing FA keys stops C1 (good) but also removes the only way to patch the wasm. H1 leaves circles with no recovery if a payout/send callback fails; H2 leaves late seats exposed to early-exit shortfall contrary to docs members may rely on. Ship H1 recovery (and decide H2) before lock.

(3) Out of scope / not verified:

Item	Why
NEAR Intents (intents.near) internals	mt_transfer failure modes, auth predecessor rules, nonce/salt rotation — not re-audited here
1Click / Confidential Intents (intents.far)	Private ledger, screening, partner-key routing
Deployed wasm ≡ this commit	No on-chain hash comparison performed this run
Current account keys on circles.contring.near / contring.near	Need near access-key listing on mainnet
Vercel env, CRON secrets, real log retention, WAF rules	Infrastructure outside the bundle
Relayer NEAR balance and live rate-limit effectiveness	Operational
	Only code review of passkey.js

Item	Why
WebAuthn PRF correctness across OS/ browser matrix	
User-attached PDF	Wrong file (SolBlaze); ignored per brief

Auditor · Contri security audit · commit 17a4a316 · circles.contring.near · 6 Nov 2026 (Africa/Lagos)
Interactive Page: https://cursor.com/canvas/store-1a6f132f-3f37-45f3-9e96-d9c8314a8e44/261c10fe-5d29-4c87-8f57-cbf47b8c8468